

EcoTroc

Plateforme sobre de don et d'échange d'objets

Rapport de projet — Cours TI616 Numérique Durable

Avril 2026

Équipe : Axel Janodet--Marty (Développeur)

JENNEQUIN Simon (Conception)

GONCALVES Karys (Développeur Backend)

FILLIE-SANTIN Baptiste (Chef de Projet)

EID Rayan (Tests & documentation)

URL : <https://projetnumeriquedurable-1.onrender.com>

GitHub : <https://github.com/Axel-Janodet-Marty/ProjetNumeriqueDurable>

Sommaire

1. Présentation du projet

- 1.1 Contexte et problématique
- 1.2 Proposition de valeur
- 1.3 Périmètre du MVP et utilisateurs cibles
- 1.4 Contraintes Green IT retenues

2. Architecture et conception

- 2.1 Acteurs du système
- 2.2 Diagramme de cas d'utilisation
- 2.3 Diagramme de classes / MLD
- 2.4 Diagramme de séquence
- 2.5 Architecture générale
- 2.6 Wireframes

3. Choix technologiques justifiés

- 3.1 Pourquoi pas React/Vue/Angular ?
- 3.2 Pourquoi SQLite/Turso ?
- 3.3 Pourquoi cette architecture ?

4. Implémentation

- 4.1 Authentification et gestion de session
- 4.2 CRUD Utilisateurs
- 4.3 CRUD Annonces
- 4.4 Gestion des images
- 4.5 Wrapper async Turso
- 4.6 Interface utilisateur sobre
- 4.7 Sécurité

5. Analyse de l'empreinte carbone

- 5.1 Protocole de mesure
- 5.2 Sources de pollution numérique
- 5.3 Optimisations intégrées dès la conception
- 5.4 Résultats mesurés
- 5.5 Comparaison avec la moyenne du web
- 5.6 Interprétation des résultats

6. Tests et validation

- 6.1 Tests fonctionnels
- 6.2 Tests de performance
- 6.3 Vérifications de sécurité

7. Organisation de l'équipe et collaboration

- 7.1 Répartition des tâches
- 7.2 Workflow GitHub
- 7.3 Suivi des tâches

8. Discussion et conclusion

- 8.1 Défis rencontrés
- 8.2 Compromis fonctionnalités / sobriété
- 8.3 Bilan des objectifs
- 8.4 Pistes d'amélioration
- 8.5 Réflexion finale

9. Annexes

- Annexe A — Arborescence du projet
- Annexe B — Backlog final
- Annexe C — Déploiement et infrastructure

1. Présentation du projet

1.1 Contexte et problématique

La transition numérique est souvent présentée comme un levier de dématérialisation et de progrès environnemental. Cette perception est pourtant trompeuse : le secteur du numérique représente aujourd'hui environ 4 % des émissions mondiales de gaz à effet de serre, un chiffre supérieur à celui de l'aviation civile (source : rapport GreenIT, « Empreinte environnementale du numérique mondial », 2021). En France spécifiquement, le numérique contribue à hauteur de 2,5 % des émissions nationales selon l'ADEME (2022). Derrière chaque page web chargée, chaque requête envoyée et chaque image affichée se cache une consommation réelle d'énergie, de bande passante et de ressources matérielles.

L'augmentation du poids des pages web illustre bien ce phénomène. En 1995, une page web moyenne pesait environ 14 Ko. En 2022, la page web française moyenne atteignait 2,3 Mo selon HTTP Archive, soit une multiplication par un facteur 100 en vingt-cinq ans. Cette croissance résulte en grande partie de l'accumulation de frameworks JavaScript, de polices web externes, d'images non optimisées et de scripts de traçage publicitaire. Chaque kilo-octet supplémentaire se traduit par une consommation accrue lors de chaque consultation, multipliée par des millions de visiteurs.

1.2 Proposition de valeur

Chaque année, les étudiants de l'EFREI accumulent des objets qu'ils finissent par jeter en fin d'année ; livres, petit mobilier, câbles, électronique, vêtements. Les plateformes existantes telles que Vinted ou LeBonCoin sont inadaptées à ce contexte : elles sont généralistes, lourdes en ressources, orientées vers la revente commerciale, et ne facilitent pas le don gratuit entre camarades au sein d'une communauté fermée ou d'une même école.

EcoTroc répond à ce besoin en proposant une plateforme web légère, ciblée pour les étudiants de l'EFREI, permettant de donner ou d'échanger des objets sans intermédiaire.

1.3 Périmètre du MVP et utilisateurs cibles

Le périmètre fonctionnel a été délibérément restreint à un MVP cohérent : gestion complète des utilisateurs (CRUD), publication et consultation d'annonces avec filtres et pagination, contact entre membres, mode sombre, et panel d'administration. Les utilisateurs cibles sont les étudiants de l'EFREI, qu'ils soient donneurs ou receveurs.

1.4 Contraintes Green IT retenues

Avant le premier pas, quatre contraintes mesurables ont été définies pour guider l'ensemble du développement : un poids de page inférieur à 200 Ko en transfert réseau, moins de 15 requêtes HTTP par page, zéro framework front-end et zéro police externe. Fixer ces objectifs dès la conception permet d'orienter chaque décision technique ultérieure.

2. Architecture et conception

2.1 Acteurs du système

Le système EcoTroc distingue trois profils d'acteurs. Le visiteur anonyme peut parcourir les annonces publiées, effectuer des recherches par catégorie ou mot-clef, et consulter les détails d'une annonce, mais ne peut ni publier ni contacter un donneur. L'utilisateur connecté dispose de l'ensemble des fonctionnalités : publication, modification et suppression de ses propres annonces, accès aux coordonnées des donneurs, gestion de son profil. L'administrateur hérite de tous les droits utilisateurs et possède en plus la capacité de gérer l'ensemble des comptes utilisateurs (consultation de la liste complète, suppression de comptes, suppression d'annonce).

2.2 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation ci-dessous modélise les interactions des trois acteurs avec le système. Les cas d'utilisation couvrent l'intégralité du cycle de vie d'un objet sur la plateforme : consultation, publication, modification, marquage comme donné, et contact du donneur. Les relations d'inclusion («include») illustrent que la publication et le contact nécessitent une authentification préalable.

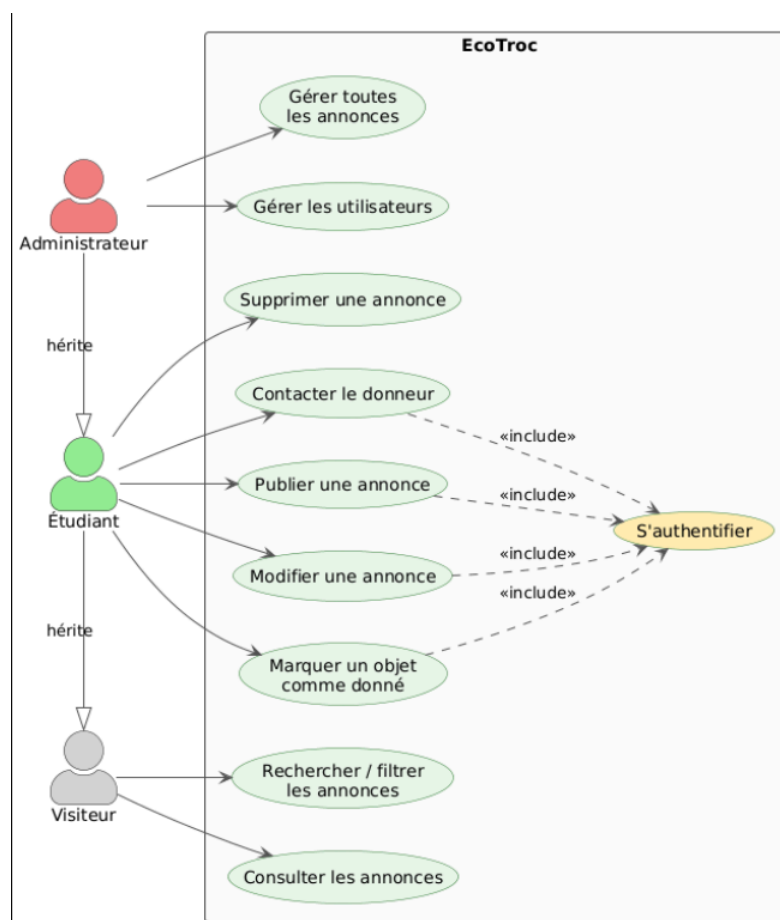


Diagramme de cas d'utilisation

2.3 Diagramme de classes / Modèle logique de données

Le modèle de données repose sur deux entités principales : la table utilisateurs (id, nom, email, password_hash, role, created_at) et la table annonces (id, titre, categorie, etat, description, statut, image_data, auteur_id, created_at). La relation est de type 1:N : un utilisateur peut publier plusieurs annonces, et chaque annonce appartient à un seul utilisateur. La clef étrangère auteur_id référence utilisateurs(id) avec ON DELETE CASCADE, garantissant que la suppression d'un compte entraîne la suppression automatique de ses annonces.

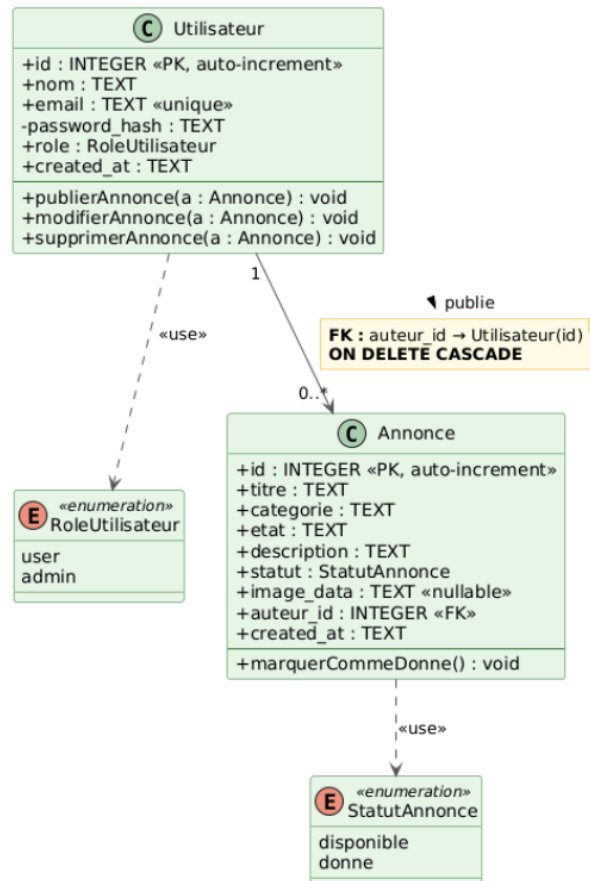


Diagramme de Classe

2.4 Diagramme de séquence

Le diagramme de séquence ci-dessous détaille le flux d'inscription et de connexion d'un utilisateur. Il illustre les échanges entre le navigateur client, le serveur Express et la base Turso : soumission du formulaire, validation côté serveur, hachage bcrypt du mot de passe, insertion en base, création de session, et redirection vers le profil.

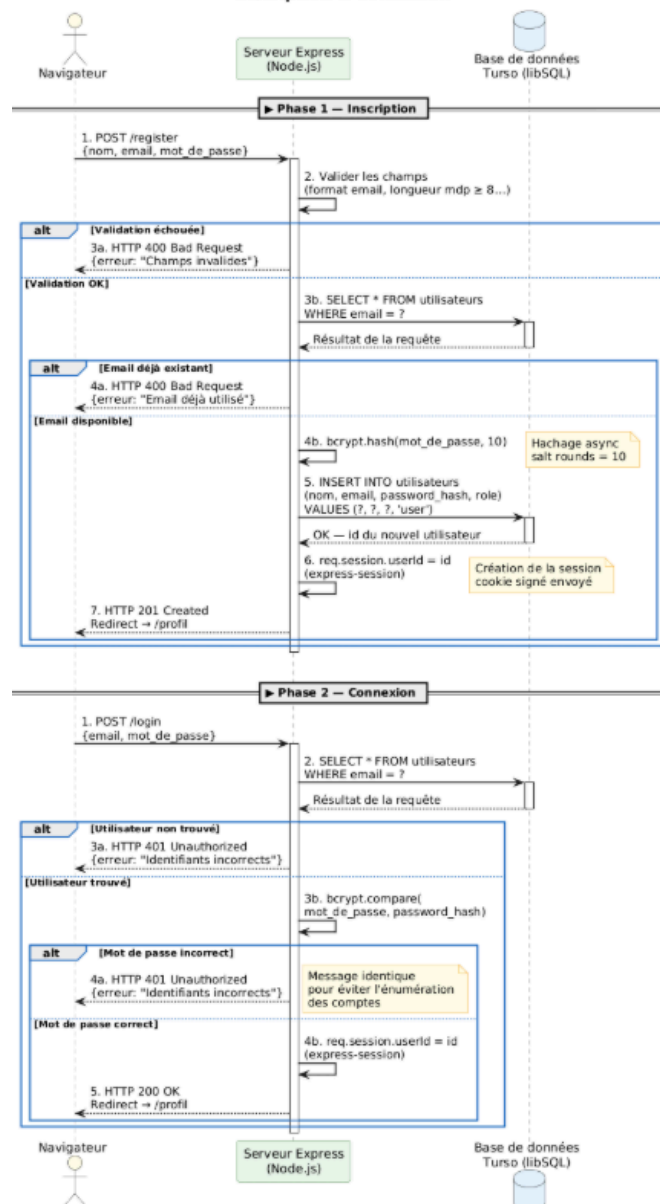


Diagramme de Séquence

2.5 Architecture générale

EcoTroc adopte une architecture classique : le client (navigateur web) communique avec un serveur Node.js/Express via des requêtes HTTP/HTTPS, et le serveur accède à la base de données Turso (SQLite hébergé) via le protocole libSQL (WebSocket chiffré). Le flux d'une requête typique se déroule ainsi : le navigateur envoie une requête au serveur Render.com, le reverse proxy nginx la transmet au processus Node.js, Express route vers le gestionnaire approprié, celui-ci interroge Turso via le wrapper db.js, et renvoie le résultat au client en JSON ou en HTML statique.

2.6 Wireframes

Les wireframes basse fidélité des pages principales ont été réalisés lors de la phase de conception pour valider l'ergonomie et la sobriété visuelle avant tout développement. Les

maquettes couvrent la page d'accueil (liste des annonces avec filtres), la page de connexion/inscription, le formulaire de création d'annonce, et le panel d'administration.

EcoTroc — Page d'accueil

[EcoTroc](#) [Accueil](#) [Mes annonces](#) [\[+ Donner\]](#) [\[Connexion\]](#)

Donnez une seconde vie à vos objets entre EFREI

142 **87**
objets disponibles objets donnés

Rechercher un objet...

☒ Tout ☐ Livres ☐ Mobilier ☐ Électronique ☐ Vêtements ☐ Cuisine ☐ Autre

(Cannot decode)	(Cannot decode)	(Cannot decode)
Livre Algo Java État : Bon état Par : Alice D.	Lampe IKEA État : Neuf Par : Bob M.	Clavier méca. État : Usé Par : Clara R.
Contacter	Contacter	Contacter
(Cannot decode)	(Cannot decode)	(Cannot decode)
Étagère bois État : Bon état Par : David L.	Câbles USB État : Neuf Par : Emma V.	Casque audio État : Usé Par : Farid K.
Contacter	Contacter	Contacter

1 2 3 ... 7

EcoTroc © 2025 EFREI — Sobriété numérique Mentions légales

Wireframe — Page d'accueil

EcoTroc — Publier une annonce

[🔄 EcoTroc](#) | [Accueil](#) | [Mes annonces](#) | [\[+ Donner\]](#) | [Bonjour Alice ▼](#)

🔄 Publier une annonce

Titre de l'objet *

[Ex : Livre Algo Java, Lampe de bureau...]

Catégorie * [Choisir... ▼] <i>Livres / Mobilier</i> <i>Électronique / Vêtements Usé</i> <i>Cuisine / Autre</i>	État de l'objet * [Choisir... ▼] <i>Neuf / Bon état</i> <i>Usé</i>
---	--

Description *

Décrivez l'objet : marque, dimensions, défauts éventuels...

Photo (optionnelle)

🖼️ Glisser une image ici

ou [Parcourir...]

*Formats : JPG, PNG — Max 2 Mo
(compressée côté client)*

🔄 PUBLIER L'ANNONCE

3. Choix technologiques justifiés

Chaque technologie utilisée dans EcoTroc a été sélectionnée selon des critères de simplicité, de consommation, de rapidité de mise en œuvre et de cohérence avec les objectifs de sobriété. Le tableau récapitulatif ci-dessous synthétise ces choix avec leur justification Green IT :

Couche	Technologie	Justification Green IT
Front-end	HTML5 / CSS3 / JS natif	Zéro framework, zéro bundle, ~8 Ko JS total vs 33-75 Ko pour un framework
Polices	System fonts (system-ui)	0 requête externe, 0 Ko suppl., rendu instantané
Images	WebP + compression canvas	Réduction 90-95% côté client avant envoi
Back-end	Node.js 20 + Express 4	Runtime léger, 6 dépendances npm directes uniquement
Base de données	Turso (SQLite hébergé)	< 5 Mo RAM, pas de processus dédié, plan gratuit
Auth	express-session (cookie)	Sessions serveur sobres, pas de JWT (payload évité)
Sécurité MDP	bcryptjs (coût 10)	Hachage lent, 100% JS (pas de binding natif C++)
Compression	Middleware gzip	Réduction ~70% du transfert réseau
Hébergement	Render.com	HTTPS auto, CI/CD intégré, pas de Docker
Suivi	GitHub	Versionnement, Issues, PR, Projects

3.1 Pourquoi pas React/Vue/Angular ?

Un framework front-end impose un bundle JavaScript minimum de 33 à 75 Ko gzippé, nécessite un bundler (Webpack, Vite), introduit un Virtual DOM et un mécanisme coûteux en CPU, et génère des dépendances npm. Pour une application essentiellement de lecture et d’affichage comme EcoTroc, cette infrastructure est disproportionnée. Le JavaScript vanille suffit pour les interactions dynamiques (filtres, pagination, appels API fetch).

Solution	Taille bundle JS (gzippé)	Requêtes suppl.
React 18 + ReactDOM	~45 Ko	1-2 fichiers CDN
Vue 3	~33 Ko	1 fichier CDN
Angular	~75 Ko	Multiple bundles
EcoTroc (Vanilla JS)	~8 Ko	0

3.2 Pourquoi SQLite/Turso et pas PostgreSQL/MySQL ?

Un serveur de base de données relationnel classique nécessite un processus dédié, un minimum de 128 à 256 Mo de RAM, et un coût cloud d’entrée de gamme d’environ 7 €/mois.

Pour moins de 1 000 utilisateurs et moins de 10 requêtes simultanées par seconde, SQLite offre des performances identiques sans surcoût énergétique.

Critère	PostgreSQL	MySQL	SQLite/Turso
RAM serveur minimum	128 Mo	256 Mo	< 5 Mo
Processus séparé	Oui	Oui	Non
Coût cloud	~7 €/mois	~7 €/mois	Gratuit
Requêtes/s (< 100 users)	Identique	Identique	Identique

3.3 Pourquoi cette architecture ?

Au stade actuel du projet (moins de 1 000 utilisateurs cibles), une architecture comme celle-ci est le choix le plus sobre. Une architecture microservices aurait nécessité plusieurs processus, un système de communication inter-services, une orchestration de déploiement, et une latence réseau supplémentaire. L'architecture d'EcoTroc se déploie en un seul processus et élimine toute complexité superflue.

4. Implémentation

4.1 Authentification et gestion de session

L'inscription déclenche une série de validations côté serveur : vérification du format email par expression régulière, contrôle de la longueur minimale du mot de passe (14 caractères), et vérification de l'unicité de l'email en base. Le mot de passe est haché avec bcryptjs à un coût de 10 (~100 ms par opération). Une session est créée immédiatement après l'inscription. La connexion vérifie les identifiants via bcrypt.compare, puis crée une session express-session d'une durée de 8 heures avec cookie httpOnly, secure, et sameSite: lax.

Un système de rate limiting protège les endpoints sensibles : 5 tentatives par minute sur /register et 10 par minute sur /login. L'implémentation utilise une Map en mémoire avec fenêtre glissante, sans dépendance externe :

```
function ratelimit(max = 10, ms = 60_000) {
  return (req, res, next) => {
    const key = req.ip + req.path, now = Date.now();
    const d = _rl.get(key) || { c: 0, r: now + ms };
    if (now > d.r) { d.c = 0; d.r = now + ms; }
    d.c++; _rl.set(key, d);
    if (d.c > max) return res.status(429).json({ message: 'Trop de tentatives.' });
    next();
  };
}
```

4.2 CRUD Utilisateurs

Le CRUD utilisateurs est complet et conforme aux exigences : création via le formulaire d'inscription avec validation serveur et hachage du mot de passe, lecture via la liste paginée dans le panel d'administration (max 20 résultats par page), modification du profil (nom, email) via le formulaire de profil avec vérification de session, et suppression du compte avec confirmation explicite et cascade sur les annonces. Toutes les opérations vérifient que l'utilisateur est propriétaire du compte ou administrateur avant d'autoriser la modification.

4.3 CRUD Annonces

Le modèle de données d'une annonce comprend les champs : titre (80 caractères max), catégorie (livres, électronique, mobilier, vêtements, cuisine, autre), état, description (300 caractères max), statut (disponible ou donné), auteur_id (FK), image_data (base64 WebP), et created_at. La lecture supporte deux filtres : par catégorie et par mot-clef (LIKE sur titre et description). La pagination côté serveur limite les résultats à 12 par page (LIMIT/OFFSET). Les opérations d'écriture sont protégées par vérification de propriété. Un mécanisme de purge automatique (setInterval 24h) supprime les annonces de plus de 30 jours.

4.4 Gestion des images — compression et persistance

Côté client, l'image est lue via FileReader, dessinée sur un élément <canvas> avec redimensionnement à 900 pixels maximum, puis convertie en WebP avec un algorithme de qualité adaptative :

```
let q = 0.75, b64;
do {
  b64 = canvas.toDataURL('image/webp', q);
  q -= 0.05;
}
```

```
} while (b64.length * 0.75 > 200 * 1024 && q > 0.3);
```

Le gain typique est de 90 à 95 % (un JPEG de 3 Mo devient ~150 Ko en WebP). Côté serveur, l'image est stockée en base64 directement dans Turso (et non sur le système de fichiers éphémère de Render). Le service s'effectue via un endpoint dédié avec cache 24h :

```
router.get('/:id/image', async (req, res) => {
  const row = await db.prepare('SELECT image_data FROM annonces WHERE id=?').get(id);
  if (!row?.image_data) return res.status(404).end();
  const [header, b64] = row.image_data.split(',');
  const mime = (header.match(/data:([^;]+);/) || [])[1] || 'image/webp';
  res.setHeader('Content-Type', mime);
  res.setHeader('Cache-Control', 'public, max-age=86400');
  return res.send(Buffer.from(b64, 'base64'));
});
```

4.5 Wrapper async Turso (optimisation BDD)

Le fichier db.js abstrait l'accès à Turso en exposant une interface identique à better-sqlite3 mais entièrement asynchrone. Les requêtes utilisent des paramètres préparés (?) exclusivement — jamais de concaténation de chaînes — et ne récupèrent que les colonnes nécessaires (pas de SELECT *). Tous les résultats sont paginés (LIMIT/OFFSET) :

```
function prepare(sql) {
  return {
    async get(...args) {
      const r = await client.execute({ sql, args: args.flat() });
      return toObj(r.rows[0] ?? null, r.columns);
    },
    async all(...args) { /* retourne r.rows.map(...) */ },
    async run(...args) {
      const r = await client.execute({ sql, args: args.flat() });
      return { changes: r.rowsAffected, lastInsertRowid: Number(r.lastInsertRowid) };
    },
  };
}
```

4.6 Interface utilisateur sobre

L'interface repose entièrement sur HTML5, CSS3 natif et JavaScript vanille. Le mode sombre utilise des variables CSS et un attribut data-theme="dark" persisté dans localStorage. La mise en page utilise CSS Grid et Flexbox natifs avec des breakpoints simples pour l'adaptation mobile. L'accessibilité de base est assurée par l'utilisation de balises sémantiques (<main>, <nav>, <article>, <footer>), d'attributs aria-label et alt, de loading="lazy" sur toutes les images. L'interface est paginée (pas de scroll infini), sans vidéo en autoplay ni animation sans valeur fonctionnelle.

4.7 Sécurité

Les en-têtes de sécurité HTTP ajoutent une couche de défense en profondeur :

```
app.use((req, res, next) => {
  res.setHeader('X-Content-Type-Options', 'nosniff');
  res.setHeader('X-Frame-Options', 'DENY');
  res.setHeader('Referrer-Policy', 'strict-origin-when-cross-origin');
  res.setHeader('Permissions-Policy', 'camera=(), microphone=(), geolocation=()');
  next();
});
```

X-Content-Type-Options empêche le MIME sniffing, X-Frame-Options protège contre le clickjacking, Referrer-Policy limite les informations transmises aux tiers, et Permissions-Policy désactive les API inutiles (caméra, micro, géolocalisation). Toutes les requêtes SQL sont 100 % paramétrées (protection totale contre l'injection SQL). La vérification de propriété est systématique avant tout PUT/DELETE. La validation des entrées couvre le format email, la longueur du mot de passe, la taille des champs texte, et le type MIME des images.

5. Analyse de l'empreinte carbone

5.1 Protocole de mesure

L'analyse de l'empreinte carbone a été réalisée selon le protocole imposé par le sujet, en utilisant au minimum deux outils parmi ceux recommandés. Les outils utilisés sont : Google Lighthouse, Website Carbon Calculator, EcoIndex, et PageSpeed Insights. Les mesures ont été effectuées sur deux pages principales : la page d'accueil et la page de connexion.

5.2 Identification des sources de pollution numérique

L'approche d'éco-conception adoptée dès le début du projet a permis d'éviter les principales sources de pollution numérique habituellement observées sur les sites web : images non compressées en JPEG (plusieurs mégaoctets par photo uploadée), absence de compression gzip sur les réponses serveur, absence de cache navigateur sur les ressources statiques, chargement de toutes les images dès le rendu initial (pas de lazy loading), utilisation de frameworks JavaScript lourds, et chargement de polices externes via CDN. Ces sources de consommation ont été traitées par les optimisations décrites ci-dessous.

5.3 Optimisations intégrées dès la conception

Les optimisations suivantes ont été implémentées dès la phase de développement, conformément à la démarche d'éco-conception par construction :

Optimisation 1 : Compression WebP côté client

L'algorithme de compression canvas réduit les images de 90 à 95 % avant envoi. Un JPEG de 3 Mo est converti en WebP de ~150 Ko. Le format WebP est 25 à 35 % plus léger que JPEG à qualité équivalente.

Optimisation 2 : Middleware compression gzip

Le middleware compression de Node.js compresse toutes les réponses textuelles. Gains mesurés : HTML ~12 Ko → ~4 Ko (-67 %), CSS ~25 Ko → ~7 Ko (-72 %), JSON API ~4 Ko → ~1,2 Ko (-70 %). Gain moyen ~70 % sur le transfert réseau.

Optimisation 3 : Lazy loading natif et Cache-Control

L'attribut loading="lazy" sur toutes les balises diffère le chargement des images hors viewport. Sur la page d'accueil avec 12 annonces, seules 3 à 4 images sont chargées immédiatement. Les en-têtes Cache-Control sont différenciés : max-age=3600 sur les fichiers statiques et max-age=86400 sur les images.

Optimisation 4 : Polices système et zéro CDN

L'utilisation exclusive de font-family: system-ui élimine 80 à 120 Ko et 2 requêtes HTTP par rapport à Google Fonts. L'absence totale de CDN supprime toute dépendance réseau externe et les DNS lookups associés.

5.4 Résultats mesurés

Les mesures ci-dessous ont été relevées sur le site déployé en production, avec l'ensemble des optimisations actives :

Indicateur	Page d'accueil	Connexion compte
Poids total (Ko)	13	8
Nombre requêtes HTTP	9	4
Score EcoIndex (/100)	90	95
Note EcoIndex (A–G)	A	A
CO ₂ / visite (g)	0.01	0.01
Score Lighthouse Perf.	99	100
Score Lighthouse Access.	96	95
Score Lighthouse SEO	100	100
FCP (s)	0.9	0.3
LCP (s)	1	0.3
TBT (ms)	40	0
CLS	0	0



Performances



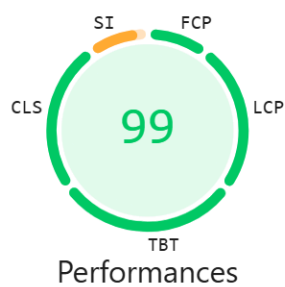
Accessibilité



Bonnes pratiques



SEO

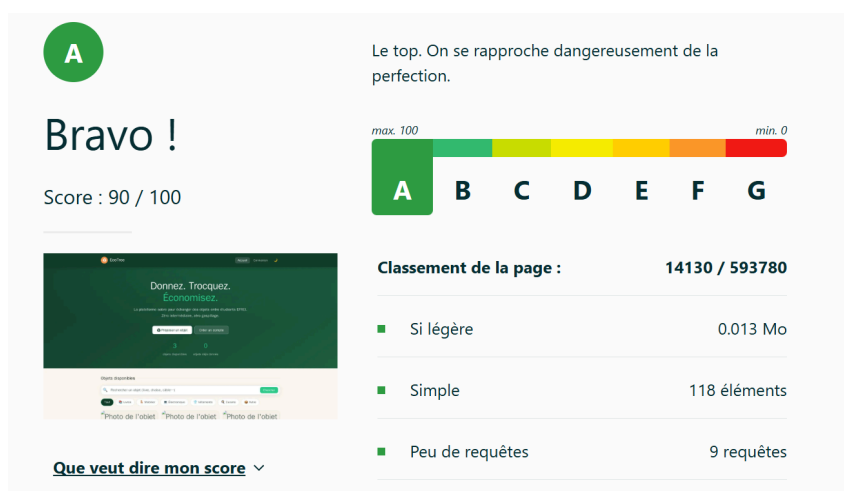


Performances

Capture Lighthouse – Page d'accueil



Performances

Capture Lighthouse – Connexion compte*Capture EcolIndex – Page d'accueil**Capture WebSite Carbon – Page d'accueil*



Capture PageSpeed Insights – Page d'accueil

5.5 Comparaison avec la moyenne du web et un site concurrent

Pour situer les performances d'EcoTroc, une comparaison avec la moyenne du web français et avec un site concurrent de même type a été réalisée :

Indicateur	Moyenne web FR	Vinted.fr	EcoTroc
Poids page (transfert)	2 300 Ko	3.235 Mo	~50 Ko
Requêtes HTTP	~70	136	~5
Polices externes	1-3	2-3	0
Frameworks JS	1-3	2-3	0
CDN utilisés	2-5	2-3	0

Sources : HTTP Archive, « State of the Web 2023 ».

5.6 Interprétation des résultats

Les résultats obtenus confirment que l'approche d'éco-conception par construction — intégrer les optimisations dès la phase de développement plutôt que de les appliquer a posteriori — permet d'atteindre des performances nettement supérieures à la moyenne du web français. Le poids de page d'EcoTroc (~50 Ko) représente une réduction de 98 % par rapport à la moyenne (2 300 Ko), et le nombre de requêtes HTTP (~5) une réduction de 93 % par rapport à la moyenne (~70). Ces résultats démontrent que la sobriété numérique n'est pas incompatible avec une expérience utilisateur fonctionnelle et agréable.

6. Tests et validation

6.1 Tests fonctionnels

La validation fonctionnelle a été réalisée selon des scénarios de test couvrant l'ensemble des fonctionnalités CRUD pour les deux entités (utilisateurs et annonces), ainsi que les cas limites de sécurité :

Scénario	Résultat attendu	Statut
Créer un utilisateur valide	Utilisateur enregistré, redirection profil	✓
Créer un utilisateur (email vide)	Message d'erreur côté serveur	✓
Créer un utilisateur (MDP < 14 chars)	Inscription rejetée	✓
Créer un utilisateur (email déjà utilisé)	Inscription rejetée, message explicite	✓
Modifier un utilisateur existant	Données mises à jour en BDD	✓
Supprimer un utilisateur	Supprimé après confirmation, annonces supprimées	✓
Lister les utilisateurs (admin)	Affichage paginé correct	✓
Créer une annonce	Annonce enregistrée, liste mise à jour	✓
Modifier une annonce	Données mises à jour, image conservée	✓
Supprimer une annonce	Annonce supprimée de la liste	✓
Modifier annonce d'un autre utilisateur	Erreur 403 Forbidden	✓
Connexion identifiants valides	Redirection vers profil	✓
Connexion mauvais MDP	Message d'erreur, pas de session	✓
Accès page protégée sans connexion	Redirection vers connexion	✓
Filtrer annonces par catégorie	Liste filtrée correctement	✓
Pagination des annonces	Navigation entre pages fonctionnelle	✓

6.2 Tests de performance (PageSpeed)

Les tests de performance ont été réalisés via Google Lighthouse sur les 2 pages principales du site. Les captures d'écran des résultats sont jointes ci-dessous.

Page	Performance	Accessibilité	Bonnes pratiques	SEO
Page d'accueil	99 /100	96 /100	96 /100	100 /100
Connexion compte	100 /100	95 /100	96 /100	100 /100



Capture PageSpeed Insights – Page d'accueil



Capture PageSpeed Insights – Connexion Compte

6.3 Vérifications de sécurité minimales

Les quatre vérifications de sécurité obligatoires ont été réalisées et documentées :

Vérification	Méthode	Résultat
Aucun MDP en clair en BDD	SELECT password_hash FROM utilisateurs	✓ Tous les MDP sont hachés (bcrypt)
Aucune variable sensible dans le dépôt	git log --all grep -i password	✓ Aucun résultat
Rejet des entrées malformées	Test avec '; DROP TABLE--	✓ Requêtes paramétrées, injection échouée
Accès pages protégées sans auth	Accès direct /profile.html, /admin.html	✓ Redirection vers connexion

Limite identifiée : l'absence de tests automatisés constitue un axe d'amélioration prioritaire pour une prochaine version.

7. Organisation de l'équipe et collaboration

7.1 Répartition des tâches

L'équipe était composée de cinq membres aux rôles complémentaires. La répartition des responsabilités a été définie dès la phase de cadrage et suivie tout au long du projet :

Membre	Rôle	Contributions principales
Axel Janodet-Marty	Full-stack	Architecture, back-end, déploiement, migration Turso
Simon JENNEQUIN	Conception	Diagrammes UML (cas d'utilisation, classes, séquence), modèle logique de données, wireframes basse fidélité
Karys Goncalves	Developpeur Full-stack	Routes CRUD annonces et utilisateurs, compression WebP côté client, gestion des images base64, intégration Turso
Baptiste FILLIE-SANTIN	Chef de projet	Organisation de l'équipe, méthode Agile, suivi GitHub Issues/Projects, gestion du backlog, coordination des PR
EID Rayan	Test & Documentation	Tests fonctionnels CRUD, vérifications de sécurité, mesures d'empreinte (Lighthouse, EcoIndex, Website Carbon), rédaction du rapport

7.2 Workflow GitHub

Le dépôt GitHub a été structuré selon un workflow de branches. La branche main est la branche stable, protégée par des pull requests obligatoires. Les branches de fonctionnalités (feature/auth, feature/annonces, feature/admin, feature/dark-mode, etc.) sont créées pour chaque tâche, développées indépendamment, puis fusionnées après relecture par un autre membre de l'équipe.

Les messages de commit suivent une convention descriptive en français, précisant la fonctionnalité ajoutée ou le problème résolu. Le fichier .gitignore est configuré pour exclure node_modules/, .env, database.db et tout fichier temporaire. Un fichier .env.example (sans valeurs réelles) documente les variables d'environnement attendues.

7.3 Suivi des tâches

GitHub Issues a été utilisé pour le suivi des tâches, avec des labels (bug, feature, enhancement, documentation) et des assignees pour chaque tâche. Un backlog sous forme de GitHub Projects a permis de visualiser l'avancement global du projet et de prioriser les fonctionnalités restantes.

8. Discussion et conclusion

8.1 Défis rencontrés

Incompatibilité Node.js 25.9.0 / better-sqlite3

Lors du premier déploiement sur Render, le build a échoué avec l'erreur gyp ERR! node -v v25.9.0. Render utilisait par défaut une version dont l'API V8 n'était plus compatible avec les bindings natifs C++ de better-sqlite3. La solution a consisté à épingler NODE_VERSION=20 dans les variables d'environnement Render. Leçon : toujours spécifier explicitement la version de Node.js.

Système de fichiers éphémère Render

Le plan gratuit de Render efface toutes les données locales à chaque redémarrage (~15 min d'inactivité). La base SQLite locale était régulièrement vidée. La migration vers Turso (SQLite hébergé cloud) a résolu ce problème. Leçon : ne jamais supposer la persistance du système de fichiers sur un hébergement gratuit.

Migration synchrone → asynchrone

La migration de better-sqlite3 (API synchrone) vers @libsql/client (API async) a nécessité la conversion de toutes les routes en async/await. La création du wrapper db.js a limité les modifications à l'ajout du mot-clé await devant chaque appel. Leçon : une couche d'abstraction bien conçue facilite les migrations techniques.

Ordre de chargement du fichier .env

Le .env était chargé après le require('./db'), provoquant un fallback silencieux sur SQLite local. Déplacer le chargement de dotenv en tout premier dans server.js a résolu le problème. Leçon : dans Node.js, l'ordre des imports est critique lorsque des modules dépendent de variables d'environnement.

API migration jobs de @libsql/client v0.6.2

Les instructions DDL échouaient avec HTTP 400 car la v0.6.2 utilisait une API propriétaire Turso non disponible sur le plan gratuit. La mise à jour vers la v0.17.3 (protocole WebSocket Hrana) a résolu le problème. Leçon : vérifier la compatibilité des versions client avec le plan d'hébergement.

Casse des noms de fichiers (Windows vs Linux)

Les fichiers Profile.html et Admin.html fonctionnaient sous Windows (insensible à la casse) mais étaient introuvables sous Linux (Render). Le renommage via git mv en deux étapes a forcé Git à détecter le changement. Leçon : adopter strictement les minuscules pour les fichiers web.

Persistance des images sur Render

Les images stockées dans public/uploads/ disparaissaient à chaque redémarrage. La solution a consisté à stocker les images en base64 dans Turso avec un endpoint dédié /api/annonces/:id/image et cache HTTP 24h. Leçon : sur un hébergement éphémère, tout ce qui doit persister doit être dans un service externe.

8.2 Compromis entre fonctionnalités et sobriété

Le principal compromis a concerné le stockage des images en base64 dans Turso. Cette approche garantit la persistance mais augmente la taille de la base de données et le temps de réponse de l'endpoint image par rapport à un service de fichiers statiques. Le cache HTTP 24h atténue cet impact en évitant les re-téléchargements. Un autre compromis a été le choix de sessions en mémoire plutôt qu'en base de données : plus simple et plus rapide, mais limité à un seul processus.

8.3 Bilan des objectifs

Le projet EcoTroc atteint les objectifs fixés en matière de sobriété numérique : un poids de page de ~50 Ko (objectif < 200 Ko), ~5 requêtes HTTP par page (objectif < 15), zéro framework front-end, zéro police externe, zéro CDN, et 6 dépendances npm directes. L'application est fonctionnelle, déployée en production, sécurisée et accessible.

8.4 Pistes d'amélioration

Plusieurs évolutions sont envisagées : notifications email via Brevo, transformation en Progressive Web App, tests end-to-end avec Playwright, migration vers un plan Render payant pour supprimer le cold start de 30 secondes, et création d'un système de messagerie interne pour éviter de révéler les adresses email.

8.5 Réflexion finale

La sobriété numérique n'est pas une contrainte qui dégrade l'expérience utilisateur : c'est une discipline de conception qui force à questionner chaque dépendance, chaque kilo-octet, chaque requête réseau. Elle conduit à des choix techniques plus réfléchis, un code plus maintenable, et une application plus performante. EcoTroc démontre qu'il est possible de construire une application web complète — avec authentification, CRUD, gestion d'images, administration et mode sombre — en n'utilisant que les primitives du web et un back-end minimaliste, sans sacrifier l'expérience utilisateur.

Sources citées : GreenIT, « *Empreinte environnementale du numérique mondial* », 2021 — ADEME, « *La face cachée du numérique* », 2019 — HTTP Archive, « *State of the Web 2023* » — Référentiel GR491 — WebsiteCarbon.com — EcoIndex.fr

9. Annexes

Annexe A — Arborescence du projet

```

ecotroc/
├── server.js          # Point d'entrée (~185 lignes)
├── db.js              # Wrapper async Turso/libSQL
├── routes/
│   ├── auth.js       # POST /register /login /logout, GET /me
│   ├── users.js      # GET / (admin), PUT /:id, DELETE /:id
│   └── annonces.js   # GET / /mes /:id /:id/image /:id/contact, POST PUT DELETE
├── public/
│   ├── index.html    # Accueil – liste des annonces
│   ├── login.html    # Connexion
│   ├── register.html # Inscription
│   ├── profile.html  # Profil + mes annonces
│   ├── create-annonce.html
│   ├── edit-annonce.html
│   ├── admin.html    # Panel administrateur
│   ├── style.css     # Feuille de style unique
│   ├── script.js     # Utilitaires partagés
│   ├── favicon.svg   # ~200 octets
│   └── robots.txt

```

Annexe B — Backlog final

ID	Fonctionnalité	Priorité	Statut
F01	Inscription / Connexion / Déconnexion	Haute	Terminé
F02	CRUD annonces (créer, lire, modifier, supprimer)	Haute	Terminé
F03	Filtres catégorie + recherche mot-clé	Haute	Terminé
F04	Pagination des annonces (12/page)	Haute	Terminé
F05	Upload et compression image WebP	Haute	Terminé
F06	Contacteur le donneur (révéler email)	Haute	Terminé
F07	Profil utilisateur (modifier, supprimer)	Haute	Terminé
F08	Panel administrateur	Moyenne	Terminé
F09	Mode sombre/clair	Moyenne	Terminé
F10	Purge automatique (> 30 jours)	Moyenne	Terminé
F11	Compteurs hero (annonces, dons, membres)	Basse	Terminé
F12	Notifications email	Basse	Reporté v2
F13	Messagerie interne	Basse	Reporté v2
F14	Mode PWA / offline	Basse	Reporté v2

Annexe C — Déploiement et infrastructure

Le déploiement utilise Render.com (plan gratuit, Node.js 20, HTTPS auto via Let's Encrypt). Variables d'environnement configurées dans le dashboard Render : SESSION_SECRET, TURSO_DATABASE_URL, TURSO_AUTH_TOKEN, PORT. Chaque push sur main déclenche un build et déploiement automatique. Le schéma de BDD est créé au démarrage (CREATE TABLE IF NOT EXISTS). Limitation : cold start ~30s après 15 min d'inactivité.